THESIS TITLE

A Thesis

Presented to the Faculty of

Lake Forest College

in Partial Fulfillment of the Requirements for the Degree of

B.A. in Computer Science

by

Sepehr Akbari

April 2026

ABSTRACT

Your abstract goes here. Make sure it sits inside the brackets. If not, your biosketch page may not be roman numeral iii, as required by the graduate school.

BIOGRAPHICAL SKETCH

Your bio-sketch goes here. Make sure it sits inside the brackets.

This document is dedicated to all Cornell graduate students.

ACKNOWLEDGEMENTS

Your acknowledgements go here. Make sure it sits inside the brackets.

TABLE OF CONTENTS

Biographical Sketch . . . iii
Dedication . . . iv
Acknowledgements . . . v
Table of Contents . . . vi
List of Tables . . . vii
List of Figures . . . viii

1 Introduction **1**

2 Gröbner Bases **2**
2.1 Polynomial Rings and Ideals . . . 3
2.2 Multivariate Division . . . 5
2.3 Buchberger's Algorithm . . . 11
2.4 Improvements to Buchberger's Algorithm . . . 17
2.4.1 Pair Elimination . . . 18
2.4.2 Selection Strategies . . . 19
2.4.3 Signature-Based Algorithms . . . 22
2.4.4 Symbolic Pre-processing . . . 24
2.5 Complexity and Computational Bounds . . . 27

3 Polynomial Models **31**

Bibliography **32**

A Algorithm Implementation **35**

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

Intro

CHAPTER 2

GRÖBNER BASES

This chapter provides a concise introduction to Gröbner bases and Buchberger's algorithm. The discussion is tailored for an audience of beginning graduate students or advanced undergraduates in mathematics and computer science. Accordingly, we assume a familiarity with standard undergraduate curricula, like calculus and linear algebra, but while a background in abstract algebra, commutative algebra, or algebraic geometry is advantageous, it is not a prerequisite for following the exposition provided here. No original work presented.

For those seeking a more comprehensive treatment, the first two chapters of [13] offer an excellent and widely recognized alternative introduction. Additional foundational perspectives can be found in [12, 24, 30], while [2] provides a more advanced theoretical framework. From a historical perspective, this work draws upon Bruno Buchberger's seminal doctoral thesis [8, 7] and his subsequent survey [10]. We also acknowledge modern optimizations that have fundamentally shaped the efficiency of the algorithm, such as the work of [19, 21, 32], which are well contextualized among other relevant works in the introductory material of [15].

The primary objective of this chapter is to establish an understanding of Buchberger's algorithm, with a particular emphasis on the role that S-pair selection strategies play in its computational performance. As the ultimate goal of this thesis is to leverage machine learning to optimize these selection processes, establishing this mathematical foundation is a critical first step.

2.1 Polynomial Rings and Ideals

The study of systems of multivariate polynomial equations is a cornerstone of both computational algebra and algebraic geometry. Consider a typical system of equations

$$\begin{cases} f_1(x, y) = x^2 - y^3 = 0 \\ f_2(x, y) = x^2 - 4y + 3 = 0 \end{cases} \tag{2.1}$$

which arise naturally across many diverse fields, like robotics [1], signal processing [25], and statistics [14, 31]. The most fundamental question regarding such a system is the *consistency* problem, which asks whether there exists an exact common solution $(x_0, y_0) \in \mathbb{C}^2$ such that all equations are simultaneously satisfied.

To approach this problem algebraically, we work within a formal structure known as a polynomial ring.

Definition 2.1.1 (Polynomial Ring)**.** The polynomial ring $R = k[x_1, \ldots, x_n]$ is the set of all polynomials in variables $x_1, \ldots, x_n$ with coefficients from a field k (typically $\mathbb{Q}, \mathbb{R}$, or $\mathbb{C}$). R is an associative algebra equipped with standard addition and multiplication operations for polynomials.

One powerful way to show that a system has no solution is to find a contradiction through a linear combination of its generators. If we can find polynomials $a_1, a_2 \in R$ such that:

$$a_1(x, y)f_1(x, y) + a_2(x, y)f_2(x, y) = 1, \tag{2.2}$$

then the system must be inconsistent. Indeed, any hypothetical solution (x_0, y_0) would force the left-hand side of (2.2) to zero, contradicting the fact that the

right-hand side is the constant 1. According to Hilbert's Nullstellensatz, over an algebraically closed field like $\mathbb{C}$, the converse is also true; as in, a system has no common zeros if and only if 1 can be expressed as a polynomial combination of the generators.

This observation shifts our focus from the equations themselves to the set of all possible polynomial combinations they can generate. This set is known as an ideal.

Definition 2.1.2 (Ideal)**.** Let $f_1, \ldots, f_s$ be polynomials in R. The ideal generated by these polynomials, denoted $\langle f_1, \ldots, f_s \rangle$, is the set

$$\langle f_1, \ldots, f_s \rangle = \left\{ \sum_{i=1}^{s} a_i f_i \mid a_i \in R \right\}. \tag{2.3}$$

Algebraically, an ideal is a sub-module of R that is closed under addition and under multiplication by any element of R. Conceptually, an ideal I represents the collection of all polynomial consequences of its generators; if a point is a zero for every generator of I, it is necessarily a zero for every polynomial in I.

The consistency of the system in (2.1) is therefore equivalent to the ideal membership problem; determining whether the constant polynomial 1 lies within the ideal $I = \langle x^2 - y^3, x^2 - 4y + 3 \rangle$. While it may not be immediately obvious for this specific example, the system (2.1) does in fact possess solutions (for instance, $(-1, 1)$), meaning $1 \notin I$. We could easily imagine more complex systems of polynomial equations, where assessing membership is far less straightforward. Thus, developing a systematic algorithmic test for such membership is the primary motivation for the theory of Gröbner bases.

2.2 Multivariate Division

In the previous section, we established that the solvability of a system of polynomial equations reduces to the ideal membership problem, which states, given a polynomial f and an ideal $I = \langle f_1, \ldots, f_s \rangle$, is $f \in I$?

In the univariate case, where $R = k[x]$, this problem is trivialized by the Euclidean division algorithm. Since $k[x]$ is a principal ideal domain, any ideal is generated by a single greatest common divisor, say g. To check if $f \in \langle g \rangle$, one simply computes the remainder of f divided by g. If the remainder is 0, then $f \in I$; otherwise, it is not. Let's illustrate this with a simple example.

Example 2.2.1. Consider the ideal $I = \langle x^2 - x \rangle$ in $\mathbb{Q}[x]$, and lets check if $h_1(x) = x^3 - x^2$ and $h_2(x) = x^3 - 1$ are elements of I. Dividing h_1 by $x^2 - x$ using polynomial long division

$$x^3 - x^2 = (x)(x^2 - x) + 0$$

yields a remainder of 0, confirming that $h_1 \in I$. Conversely, for h_2, we have

$$x^3 - 1 = (x + 1)(x^2 - x) + (x - 1)$$

with a non-zero remainder of $x - 1$, indicating that $h_2 \notin I$. This is a direct and trivial application of the division algorithm in one variable.

However, generalizing this logic to the multivariate ring $k[x_1, \ldots, x_n]$ introduces complications. Consider another example.

Example 2.2.2. Let $I = \langle x^2y - x, x^2 + y^3 \rangle$ in the ring $\mathbb{Q}[x, y]$, and let's check if $h_3(x, y) = x^2y + y^4$ is an element of I. Here, I is generated by two polynomials, its elements can be expressed as combinations of the generators: $a_1(x^2y - x) +$

$a_2(x^2 + y^3)$ for some $a_1, a_2 \in \mathbb{Q}[x, y]$. This example follows the same principle as before, but considering that a_1 and a_2 can be any polynomials in two variables, the search space is vastly larger. Other than impractical trial and error, there is no straightforward method to determine if such a_1 and a_2 exist that satisfy the equation $h_3 = a_1(x^2y - x) + a_2(x^2 + y^3)$.

Our goal is to apply our solution for Example 2.2.1 to Example 2.2.2 by developing a multivariate division algorithm. However, two main issues arise in this generalization.

Firstly, in Example 2.2.1, there only exists one polynomial divisor to be used in the division algorithm, but in Example 2.2.2, there are two generators in the ideal, leading to multiple possible divisors to choose from at each step. We can overcome this problem by simply choosing one of the divisors in each step and keeping track of a quotient for each divisor. The final result in the univariate case is expressed as $f = aq + r$, where a is the quotient, q is the divisor, and r is the remainder. In the multivariate case with multiple divisors, we can express the result as $f = a_1q_1 + \cdots + a_sq_s + r$, where each a_i is the quotient corresponding to divisor q_i. This expression still captures the essence of the division algorithm, but now accommodates multiple divisors.

Secondly, notice that in the division algorithm we divide the leading term of the dividend by the leading term of the divisor, giving importance to what we consider the leading term, or the largest term in a polynomial. In the univariate case, this is straightforward since there is a natural ordering of terms by degree ($x^2 > x$). In the multivariate case, however, there is no inherent way to order terms like x^2y and xy^2. To address this, we need to follow a systematic rule for ordering monomials in multiple variables to ensure consistency in determining

leading terms. We denote a monomial $x_1^{\alpha_1} \dots x_n^{\alpha_n}$ as x^α, where $\alpha \in \mathbb{Z}^n_{\geq 0}$.

Definition 2.2.1 (Monomial Ordering)**.** A monomial ordering on $k[x_1, \dots, x_n]$ is a relation $>$ on the set of monomials such that $>$ is a total ordering (any two monomials are comparable), $>$ is compatible with multiplication, as in, if $x^\alpha > x^\beta$, then $x^\alpha x^\gamma > x^\beta x^\gamma$ for all γ, and $>$ is a well-ordering (every non-empty set of monomials has a least element).

The conditions of monomial ordering are set to guarantee consistency and termination of the division algorithm. The total ordering condition ensures that we can always identify a leading term in any polynomial. The compatibility condition guarantees that the ordering behaves predictably under multiplication, and the well-ordering condition is specially critical for computer science applications, as it guarantees that any algorithm reducing the size of a polynomial based on this order will eventually terminate. While there are infinitely many such orderings, two are very frequently used in computational algebra; lexicographic order (lex) and graded reverse lexicographic order (grevlex).

Definition 2.2.2 (Lexicographic Order)**.** Let $\alpha, \beta \in \mathbb{Z}^n_{\geq 0}$. The lexicographic order is defined by $x^\alpha >_{lex} x^\beta$ if the leftmost non-zero entry of the vector difference $\alpha - \beta$ is positive.

Intuitively, Lex behaves like a dictionary sort. For variables $x > y > z$, we have $x > y^5 z^3$ because the x-exponent dominates. On the other hand, $y^5 z^3 > y^4 z^{10}$ because the y-exponent is larger.

Definition 2.2.3 (Graded Reverse Lexicographic Order)**.** Graded reverse lexicographic order is defined by $x^\alpha >_{grevlex} x^\beta$ if $|\alpha| > |\beta|$, or if $|\alpha| = |\beta|$ and the rightmost non-zero entry of $\alpha - \beta$ is negative.

Grevlex compares total degree first. For example, $x > y > z$, $y^2 > xz$ since $2 > 1$. If total degrees are equal, it favors monomials that have fewer variables from the end of the ordering. Thus, $xy > xz$ because the y-exponent is larger than the z-exponent. For computation, grevlex is often more efficient than lex as it tends to keep coefficients smaller.

With a fixed monomial order, we can define the leading term of a polynomial f, denoted $\mathrm{LT}(f)$, as well as its coefficient $\mathrm{LC}(f)$ and monomial $\mathrm{LM}(f)$.

Given a monomial order and a set of divisors $F = \{f_1, \ldots, f_s\}$, we can formulate a multivariate polynomial long division algorithm with the goal to write f as

$$f = a_1 f_1 + \cdots + a_s f_s + r$$

where no term in the remainder r is divisible by any $\mathrm{LT}(f_i)$. Algorithm 1 outlines this process.

Definition 2.2.4 (Reduction)**.** When a polynomial h is divided by a set of polynomials $F = \{f_1, \ldots, f_s\}$ using the multivariate division algorithm, the resulting polynomial r is written as $\mathrm{reduce}(h, F)$ and is said that h reduces to r.

Algorithm 1: Multivariate Division Algorithm

Input $h, F = \{f_1, \ldots, f_s\}, >$

Output r

$p \leftarrow h, r \leftarrow 0$

while $p \neq 0$ **do**

if $\exists i$ s.t. $LT(f_i) \mid LT(p)$ **then**

$p \leftarrow p - \frac{LT(p)}{LT(f_i)} f_i$

else

$r \leftarrow r + LT(p), \quad p \leftarrow p - LT(p)$

end if

end while and **return** r

We denote the result of this process as $r = \text{reduce}(h, F)$. If $\text{reduce}(h, F) = 0$, we have explicitly constructed coefficients a_i such that $h = \sum a_i f_i$, proving that $h \in I$. However, unlike the univariate case, a non-zero remainder does *not* prove that $h \notin I$. The output of the division algorithm depends heavily on the order in which the divisors f_i are listed.

To demonstrate this, let us apply Algorithm 1 to Example 2.2.2, using the Lexicographic order with $x > y$. In this example, $f_1 = x^2y - x$ and $f_2 = x^2 + y^3$, and we want to check if $h = x^2y + y^4$ is in the ideal I. First, we list the divisors as $I = \langle f_1, f_2 \rangle$, so the algorithm checks f_1 first. In this case, the final result is $r = x + y^4 \neq 0$, suggesting that $h \notin I$. If we then list the divisors as $I = \langle f_2, f_1 \rangle$. The algorithm checks f_2 first, in which case the final result is $r = 0$, indicating that $h \in I$. Since we found a remainder of 0 in the second case, we have proven that $h \in I$. This example highlights that the standard multivariate division algorithm

is not sufficient for the ideal membership problem because the remainder is not unique. Have in mind, that this is a simple example with only two generators, and in practice, ideals can have many generators and the orderings can be much more complex.

The failure of the division algorithm arises because the leading terms of the generators $\{f_1, \ldots, f_s\}$ may not cover all the leading terms of the ideal I. Cancellation can occur between generators during linear combination, producing a new polynomial in I with a leading term not divisible by any $\mathrm{LT}(f_i)$. To resolve this, we could try to start the division process with a divisor that is an element of I itself. However if the leading term of this element is not divisible by any $\mathrm{LT}(f_i)$, we cannot proceed with the division. Thus, we need a generating set of I such that every element of I has its leading term divisible by some leading term of the generating set. A Gröbner basis is a generating set with this property.

Definition 2.2.5 (Gröbner Basis)**.** Let there be a fixed monomial order. A finite subset of generators $G = \{g_1, \ldots, g_t\} \subset I$ is a Gröbner basis for a non-zero ideal I if

$$\langle \mathrm{LT}(g_1), \ldots, \mathrm{LT}(g_t) \rangle = \langle \{\mathrm{LT}(f) \mid f \in I\} \rangle$$

This definition implies two equivalent properties that resolve the issues discussed above:

(i) $f \in I \iff \mathrm{reduce}(f, G) = 0$

(ii) $\mathrm{reduce}(f, G)$ is unique $\forall f \in R$ regardless of the order of g_i.

Property (i) directly answers Example 2.2.2, as if we could find a Gröbner basis G for I, we could simply compute $\mathrm{reduce}(h_3, G)$ to determine if $h_3 \in I$ without taking account of the order of generators. Property (ii) ensures that the multivariate division algorithm behaves consistently when using a Gröbner basis,

making it a reliable tool for computations in polynomial ideals. This consistency and reliability makes Gröbner bases a widely used tool in computational algebraic geometry and computer algebra systems.

2.3 Buchberger's Algorithm

While the existence of Gröbner bases is theoretically guaranteed, their practical utility depends on our ability to compute them explicitly. In 1965, Bruno Buchberger introduced the fundamental algorithm for constructing these bases in [8, 7]. The core of his method relies on detecting and repairing "ambiguities" where the multivariate division algorithm fails to be unique.

The primary obstruction to uniqueness in multivariate division is the cancellation of leading terms. If we have two polynomials $f, g \in I$, it is possible that a linear combination $af + bg$ causes the leading terms to cancel, revealing a new leading term of lower order that was previously hidden. Buchberger identified that we do not need to check all random linear combinations. We only need to check the minimal cancellation between the leading terms of pairs of generators.

Definition 2.3.1 (S-polynomial)**.** Let $f, g \in k[x_1, \ldots, x_n]$ be non-zero polynomials. Let $x^\gamma = \text{lcm}(\text{LM}(f), \text{LM}(g))$ be the least common multiple of their leading monomials. The S-polynomial of f and g is defined as

$$S(f, g) = \frac{x^\gamma}{\text{LT}(f)} \cdot f - \frac{x^\gamma}{\text{LT}(g)} \cdot g$$

The S-polynomial is constructed specifically to cancel the leading terms of f and g. It is very likely that this cancellation reveals a new leading term in the

ideal that is not divisible by either LT(f) or LT(g). In other words, if this cancellation results in a polynomial that cannot be reduced to zero by the current set of generators [1], we have found a new polynomial that needs to be added to our generating set to ensure it covers all leading terms of the ideal.

Now suppose $f \in I$, then by Definition 2.1.2, f can be expressed as

$$f = \sum_{i=1}^{s} a_i g_i$$

for some $a_i \in R$ and $g_i \in G$. Observe that each term in this sum has a lead term LT($a_i g_i$) = LT(a_i)LT(g_i), which is divisible by LT(g_i). The only way that f has a leading term not divisible by any LT(g_i) is if there is cancellation among these leading terms. We can trace this cancellation back to pairs of generators g_i, g_j whose leading terms cancel in the sum. The minimal such cancellation is exactly the S-polynomial $S(g_i, g_j)$, and by our assumption that $f \in I$, we must have $S(g_i, g_j) \in I$ as well. Therefore, if G is a Gröbner basis, by property *(i)* from Definition 2.2.5, we must have reduce($S(g_i, g_j), G$) = 0 for all pairs i, j. Conversely, if this condition holds for all pairs, then any polynomial $f \in I$ cannot have a leading term that escapes divisibility by some LT(g_i), because any such escape would trace back to a non-zero remainder from some S-polynomial. This leads us to Buchberger's Criterion.

Theorem 2.3.1 (Buchberger's Criterion)**.** Let $G = \{g_1, \ldots, g_s\}$ be a generating set for an ideal I. Then G is a Gröbner basis for I if and only if:

$$\text{reduce}(S(g_i, g_j), G) = 0$$

for all pairs $1 \leq i < j \leq s$.

[1] It is fascinating that the S-polynomial reveals all potential ambiguities or problems with a generating set.

This theorem transforms the definition of a Gröbner basis from an abstract property about infinite sets (all polynomials in I) into a finite computational check. Buchberger's algorithm is a direct application of the criterion above, which computes a Gröbner basis of I from any starting generating set of I, by iteratively checking all S-polynomials of our current generating set. If any S-polynomial reduces to a non-zero remainder, we force them to reduce to zero, by adding their reduction to our generating set. This process continues to reveal even more S-polynomials to check, until all S-polynomials reduce to zero. At this point our generating set is a Gröbner basis.

An outline of Buchberger's algorithm is provided in Algorithm 2. There are three main functions used in the algorithm. We assume we have a function that returns a pair from the set of pairs P. This selection could be as simple as treating the pair set as a queue data structure and returning the first element at each iteration. We also need a function *reduce* that implements polynomial long division and returns the remainder, as defined in Algorithm 1. Finally, an *update* function is needed to update the set of pairs P when a new generator is added to the basis, defined as

$$\text{update}(P, G, r) = P \cup \{(r, h) \mid h \in G\}$$

where r is the new polynomial added to the basis G, and h iterates over all existing generators in G.

Algorithm 2: Buchberger's Algorithm

Input $F = \{f_1, \ldots, f_s\}$

Output Gröbner basis G for $I = \langle F \rangle$

$G \leftarrow F$

$P \leftarrow \{(f_i, f_j) \mid f_i, f_j \in G, f_i \neq f_j\}$

while $P \neq \emptyset$ **do**

    $(f_i, f_j) \leftarrow \text{select}(P)$

    $P \leftarrow P \setminus \{(f_i, f_j)\}$

    $r \leftarrow \text{reduce}(S(f_i, f_j), G)$

    if $r \neq 0$ **then**

        $P \leftarrow P \cup \{(r, h) \mid h \in G\}$

        $G \leftarrow G \cup \{r\}$

    end if

end while

return G

The termination of the Buchberger algorithm is not immediately obvious, as each iteration can potentially add new polynomials to G, leading to more S-polynomials to check, and thus an unbounded loop. However, the algorithm is guaranteed to terminate by considering the monomial ideal generated by the leading terms of the elements in G. Each time a non-zero remainder $r = \text{reduce}(S(f_i, f_j), G)$ is added to G, its leading term $\text{LT}(r)$ is not divisible by any of the leading terms of the current generators. Consequently, the new monomial ideal $\langle \text{LT}(G \cup \{r\}) \rangle$ strictly contains the previous ideal $\langle \text{LT}(G) \rangle$. This process cre-

ates a strictly ascending chain of monomial ideals

$$\langle \mathrm{LT}(g_1) \rangle \subsetneq \langle \mathrm{LT}(g_1), \mathrm{LT}(g_2) \rangle \subsetneq \langle \mathrm{LT}(g_1), \mathrm{LT}(g_2), \mathrm{LT}(g_3) \rangle \subsetneq \dots$$

According to the Ascending Chain Condition (ACC), every ascending chain of ideals in the polynomial ring $k[x_1, \dots, x_n]$ over the field k (or any Noetherian ring) must stabilize. In the context of monomial ideals, this is a direct consequence of Dickson's Lemma, as shown in [13], which states that every monomial ideal is finitely generated. Therefore, the chain cannot increase indefinitely, ensuring the algorithm terminates in a finite number of steps.

As proved in [13], Algorithm 2 finds a Gröbner basis for any ideal I. However, in many cases the output of Buchberger's algorithm often contains redundant elements.

Example 2.3.1. Consider the ideal $I = \langle x^2, xy + y, x^2y + z \rangle$ in $\mathbb{Q}[x, y, z]$. Using Grevlex ordering, Buchberger's algorithm produces the Gröbner basis

$$G = \left\{ x^2,\ xy + y,\ x^2y + z,\ y,\ -z \right\}$$

Notice that $\mathrm{LT}(x^2y + z) = x^2y$ is divisible by $\mathrm{LT}(x^2) = x^2$. So $x^2y + z$ is redundant in the sense that removing it from G does not change the ideal generated by the leading terms.

To address this redundancy, we can remove these redundant polynomials from G without losing the Gröbner basis property, to obtain a minimal Gröbner basis.

Definition 2.3.2 (Minimal Gröbner Basis)**.** A Gröbner basis G is minimal if $\mathrm{LT}(g_i)$ does not divide $\mathrm{LT}(g_j)$ for any $i \neq j$.

Example 2.3.2. Continuing from Example 2.3.1, we can rewrite the Gröbner basis G as

$$G_{minimal} = \left\{ x^2,\ y,\ -z \right\}$$

which is a minimal Gröbner basis for the ideal I. Observe that removing the redundant polynomials did not change the ideal generated by the leading terms, as $\langle x^2, y, -z \rangle = \langle x^2, xy + y, x^2y + z \rangle$.

A powerful property in mathematics is uniqueness, which allows us to identify canonical representatives for equivalence classes. In the context of Gröbner bases, we can further refine the concept of minimality to achieve uniqueness among all Gröbner bases for a given ideal with a fixed monomial order.

Definition 2.3.3 (Reduced Gröbner Basis)**.** A Gröbner basis G is reduced if, for all $g \in G$ no monomial appearing in g is divisible by any $\mathrm{LT}(h)$ for $h \in G \setminus \{g\}$, and, $\mathrm{LC}(g) = 1$ (monic polynomials).

Example 2.3.3. Continuing from Example 2.3.2, we can further reduce the minimal Gröbner basis $G_{minimal}$ to obtain

$$G_{reduced} = \left\{ x^2,\ y,\ z \right\}$$

which is a reduced Gröbner basis for the ideal I. Note that we changed $-z$ to z to make it monic.

Similar to many powerful tools in mathematics, like reduced row echelon form in linear algebra, the significance of reduced Gröbner bases lies in their uniqueness. For any ideal I and a fixed monomial order, there exists a unique reduced Gröbner basis. Most implementations of Buchberger's algorithm, including the *gb* function in *Macaulay2* [22], automatically return this reduced

Gröbner basis through the same process shown in Examples 2.3.1, 2.3.2, and 2.3.3.

2.4 Improvements to Buchberger's Algorithm

The naive implementation of Buchberger's Algorithm (Algorithm 2) guarantees termination, but its efficiency is usually improved for practical applications.

One of the computational bottleneck in Algorithm 2 is computing reduce($S(f_i, f_j), G$). The most significant optimizations focus on improving the choice of divisor in Algorithm 1. It is typically most efficient to choose the smallest LT(f_i) available in each iteration. This strategy minimizes the size of intermediate polynomials during reduction, leading to faster computations. Additionally, we modify Algorithm 1 to perform a full tail reduction, ensuring that no term in the remainder r is divisible by any LT(f_i).

The most important implementation choices occur in the *select* and *update* functions. Empirically, the vast majority of S-polynomials reduce to zero. These zero reductions are computationally expensive but mathematically insignificant, as they tell us that the current basis is already sufficient for that specific pair, adding no new information. The central challenge in optimizing Buchberger's algorithm is therefore, preemptively detecting which S-polynomials will reduce to zero so we can skip them entirely, and deciding which surviving pair to process next to keep intermediate coefficients small and induce future zero reductions as early as possible.

2.4.1 Pair Elimination

The most effective optimization is to remove redundant S-polynomials $S(g_i, g_j)$ for the eventually computed Gröbner basis $G = \{g_1, \ldots, g_s\}$ before they are ever computed. Observe that in Theorem 2.3.1, if we can theoretically guarantee that an S-polynomial reduces to zero using the existing elements of G, we can discard the pair immediately without actually reducing them. There are two fundamental criteria for elimination, often combined into the Gebauer-Möller rules [19].

Lemma 2.4.1 (Product Criterion)**.** Let $f, g \in G$. If their leading monomials are relatively prime, i.e.,

$$\operatorname{lcm}(\operatorname{LM}(f), \operatorname{LM}(g)) = \operatorname{LM}(f) \cdot \operatorname{LM}(g)$$

then $S(f, g)$ reduces to zero modulo $\{f, g\}$.

Proof. If the leading monomials share no variables, the cancellation in the S-polynomial corresponds to the trivial relation $g \cdot f - f \cdot g = 0$. This symmetry ensures the S-polynomial reduces to zero. □

Lemma 2.4.2 (Chain Criterion)**.** Consider three generators $f, g, h \in G$. If $\operatorname{LM}(h)$ divides the $\operatorname{lcm}(\operatorname{LM}(f), \operatorname{LM}(g))$ then

$$S(f, g) = \frac{\operatorname{lcm}(\operatorname{LM}(f), \operatorname{LM}(g))}{\operatorname{LT}(f)} \cdot S(f, h) - \frac{\operatorname{lcm}(\operatorname{LM}(f), \operatorname{LM}(g))}{\operatorname{LT}(g)} \cdot S(g, h)$$

and so we have already processed the pairs (f, h) and (g, h), then the pair (f, g) is redundant.

Intuitively, the S-polynomial $S(f, g)$ accounts for the ambiguity between f and g. If h sits between them in the divisibility lattice, the ambiguity is already resolved by the interactions $f \leftrightarrow h$ and $h \leftrightarrow g$.

The Gebauer-Möller rules in [19] combine these two criteria, originally shown in [9], to efficiently prune the set of pairs P during the *update* step of Buchberger's algorithm.

2.4.2 Selection Strategies

At any given step of Buchberger's algorithm, the set of pending pairs P may contain large numbers of candidates. The order in which we process these pairs, in the *select* function, is often the deciding factor between an efficient computation and an infeasible one. A simple implementation of *select* might treat P as a queue, processing pairs in the order they were created. This is an inefficient strategy, and although the correctness of the algorithm is unaffected, many additional redundant generators may be created, leading to a combinatorial explosion in the number of pairs to consider.

The governing heuristic for selection in reduction algorithms like the Euclidean algorithm and Buchberger's algorithm is to choose the small elements first. We aim to process pairs that are likely to result in simple polynomials, those with low degree or few terms, or more mathematically described as those pairs with their lead monomials having small least common multiple in the monomial order or in total degree. These "small" pairs are more likely to reduce to zero quickly, preventing the growth of intermediate expressions.

Four standard selection strategies are commonly used, each distinguished by their definition of "small." In Algorithm 2, even our naive implementation of *select* using a queue can be viewed as a specific strategy; selecting the pair that was created first. So for selecting a pair $(f_i, f_j) \in P$, we select the pair with

minimal i among all pairs with minimal j.

Definition 2.4.1 (Order-Based Selection)**.** Utilizes the monomial order > already fixed for the ring, and selects (f_i, f_j) such that $\text{lcm}(\text{LM}(f_i), \text{LM}(f_j))$ is minimal with respect to >.

Definition 2.4.2 (Degree-Based Selection)**.** Selects the pair (f_i, f_j) such that $\deg(\text{lcm}(\text{LM}(f_i), \text{LM}(f_j)))$ is minimal.

Definition 2.4.1 was first proposed by Buchberger in [7], and is often called the *Normal* strategy. In the context of the graded reverse lexicographic order (grevlex), this strategy prioritizes pairs with the lowest total degree. In the event of a tie in total degree, it selects the pair that is smallest in the reverse lexicographic sense. Final ties are typically resolved by creation order (the queue strategy). Both strategies aim to create a breadth-first traversal of the ideal, resolving all ambiguities at degree d before moving to degree $d + 1$. This is the standard default for many computer algebra systems because it naturally controls the growth of intermediate polynomials.

However, both the Normal and Degree-based strategies falter in non-homogeneous ideals because they rely on the apparent degree. In these cases, leading term cancellations can cause the degree of a polynomial to drop, masking the true computational complexity of its ancestry.

Example 2.4.1. Consider the polynomial $f = x^{10} + x$ with $\deg(f) = 10$. If a reduction step subtracts x^{10}, the resulting polynomial is $r = x$, which has an apparent degree of 1. Despite its small size, r originated from a degree 10 polynomial. If the algorithm treats r as a degree-1 polynomial, it may prematurely prioritize S-polynomials involving r, inadvertently reintroducing high-degree terms that were previously cancelled and causing a computational explosion.

The concept of r "remembering" its high-degree origin is formalized in the *Sugar Cube* strategy in [21]. This strategy treats the degree of a non-homogeneous polynomial as a "phantom degree" to emulate the efficiency of homogeneous computations.

Definition 2.4.3 (Sugar Cube Selection)**.** The sugar of a polynomial f is initialized as $\text{sugar}(f) = \deg(f)$ for all input polynomials. For any subsequent polynomial h generated during the algorithm (e.g., via S-polynomials or reductions), its sugar is defined recursively.

Given f_i and f_j, let $L = \text{lcm}(\text{LM}(f_i), \text{LM}(f_j))$. We define the relative sugar of each component as

$$\sigma_i = \text{sugar}(f_i) + \deg\left(\frac{L}{\text{LM}(f_i)}\right)$$
$$\sigma_j = \text{sugar}(f_j) + \deg\left(\frac{L}{\text{LM}(f_j)}\right)$$

The sugar of the resulting S-polynomial $S(f_i, f_j)$ is then simply

$$\text{sugar}(S(f_i, f_j)) = \max(\sigma_i, \sigma_j)$$

If f is reduced by g to obtain $h = f - mg$, where m is a monomial, the sugar is updated as

$$\text{sugar}(h) = \max(\text{sugar}(f), \text{sugar}(g) + \deg(m))$$

The algorithm prioritizes critical pairs (f_i, f_j) that minimize the sugar of their associated S-polynomial.

Sugar selection prevents the algorithm from being tricked by cancellations. It treats the "phantom" high-degree terms as if they were still present, ensuring

that we do not process complex relationships simply because they appear simple on the surface. For difficult, non-homogeneous benchmark problems, Sugar is consistently known to be the most robust strategy.

There are also several adversarial and baseline strategies used to benchmark the performance of selection strategies. The *Random* strategy provides a stochastic baseline by selecting a pair uniformly at random from the set of critical pairs. Structural baselines include *Last*, which treats the pair set as a stack by selecting the pair (f_i, f_j) with the maximal indices j and then i. Adversarial strategies designed to induce degree growth include *Codegree*, which prioritizes the maximal total degree of the $\mathrm{lcm}(\mathrm{LM}(f_i), \mathrm{LM}(f_j))$, and *Strange*, which selects the pair with the largest lcm according to the specific monomial ordering. Finally, the *Spice* strategy acts as a direct stress test by selecting the pair with the largest sugar degree, representing the maximum degree the S-polynomial would achieve under homogenization. These strategies serve as essential benchmarks for validating that prioritizing high-degree elements leads to computational infeasibility.

2.4.3 Signature-Based Algorithms

The pair elimination criteria detailed in Section 2.4.1 are restricted to the leading monomials of the current basis. While computationally efficient, these criteria are inherently myopic, as they neglect the algebraic history of the generators. Signature-based algorithms, such as Faugère's F_5 [16], mitigate this by tracking the symbolic origin of each polynomial throughout the reduction process.

Formally, any polynomial p in the ideal $I = \langle f_1, \ldots, f_m \rangle \subset R$ can be lifted to the free module R^m. Let $\{\mathbf{e}_1, \ldots, \mathbf{e}_m\}$ be the standard basis of R^m, where each $\mathbf{e}_i$

maps to the input generator f_i. Any $p \in I$ is the image of a module element $\alpha \in R^m$ under the map $\pi : R^m \to I$, defined by

$$\pi(\alpha) = \sum_{k=1}^{m} u_k f_k = p$$

This module-theoretic framework allows for the tracking of synergies and redundancies through the concept of signatures.

The *signature* of a polynomial p, denoted $\sigma(p)$, is defined as the leading term of its module representative α with respect to a module term order (such as position-over-coefficient (POT)). Specifically, if $\mathrm{LT}(u_i)\mathbf{e}_i$ is the maximal term in α under the chosen order, then

$$\sigma(p) = \mathrm{LT}(u_i)\mathbf{e}_i$$

where i denotes the index of the leading module component. By augmenting polynomials with these signatures, algorithms can identify and discard S-polynomials that correspond to known syzygies of the initial generators, thereby preempting redundant reductions.

In the classic Buchberger algorithm, an S-polynomial is computed and reduced; if the result is zero, it is discarded. An algebraic reduction to zero implies the discovery of a syzygy (a relation between the generators). Signature algorithms operate on the principle that these zero reductions can be predicted. If a pair of polynomials implies a relation matching a trivial syzygy (algebraically equivalent to $f_i f_j - f_j f_i = 0$), the algorithm detects this via signature comparison. This strategy is formalized in the F_5 algorithm.

Definition 2.4.4 (F_5 Criterion)**.** Let $p_i, p_j \in I$ be polynomials with signatures $\sigma(p_i)$ and $\sigma(p_j)$. An S-polynomial $S(p_i, p_j)$ is deemed redundant and can be discarded under the F_5 criterion if its signature is a multiple of the signature

of an existing basis element. Specifically, the algorithm maintains the invariant that it only processes pairs with distinct, non-trivial signatures. If the signature of a proposed reduction suggests a dependency on a known principal syzygy (the trivial commutation of generators), the pair is rejected without arithmetic reduction.

This criterion allows the algorithm to distinguish between useful reductions that expand the basis and useless reductions that merely rediscover known algebraic identities. The efficiency gains are most pronounced for specific classes of ideals.

For regular sequences, ideals where the generators exhibit no algebraic dependencies other than the trivial principal syzygies, the F_5 algorithm guarantees the computation of a Gröbner basis without performing a single zero reduction. Given that zero reductions constitute the primary computational bottleneck in standard Buchberger variants, often consuming upwards of 90% of total runtime, the elimination of these redundant steps offers a massive theoretical and practical speedup.

2.4.4 Symbolic Pre-processing

Before engaging the computationally intensive machinery of Buchberger's algorithm or F_5, it is imperative to simplify the input ideal through symbolic preprocessing. These preliminary steps, while computationally inexpensive relative to the basis construction, significantly improve the numerical conditioning and algebraic structure of the problem.

Because the complexity of computing a Gröbner basis is doubly exponential in the number of variables n. Therefore, strictly reducing the dimension of the polynomial ring is the single most effective optimization available. If the ideal I contains linear generators (polynomials of total degree 1), these imply direct dependencies between variables. We can exploit this by performing Gaussian elimination on the linear subsystem of generators. For a linear generator $f = c_1x_1 + c_2x_2 + \cdots + c_nx_n + d$ we select a variable x_k with a non-zero coefficient to serve as the pivot. We solve for x_k and substitute the resulting affine expression into the remaining generators f_j for $j \neq i$. This substitution defines a ring isomorphism that effectively projects the ideal into a polynomial ring of dimension $n - 1$. This process is repeated iteratively until all linear generators have been eliminated.

Once the dimension is minimized, we must address the stability of the generators. Affine Gröbner basis computations often suffer from degree drops, where the cancellation of leading terms results in a polynomial of significantly lower degree. These lower-degree terms can disrupt the selection strategy of the algorithm. While the Sugar strategy attempts to mitigate this by simulating homogeneity through virtual degree tracking, *explicit homogenization* offers superior stability by embedding the problem into projective space. We introduce a homogenization variable t to map the ideal from $k[x_1, \ldots, x_n]$ to the graded ring $S = k[x_1, \ldots, x_n, t]$. For each generator f, we compute its homogeneous form f^h by

$$f^h(x_1, \ldots, x_n, t) = t^{\deg(f)} \cdot f\left(\frac{x_1}{t}, \ldots, \frac{x_n}{t}\right)$$

Working in the projective ring S ensures that all polynomials in the current basis are homogeneous. Consequently, the degree of an S-polynomial is strictly determined by the degrees of its parents, eliminating the unpredictability of affine

degree falls. Once the homogeneous Gröbner basis G^h is computed, the affine solution is recovered by the de-homogenization map (setting $t = 1$), which corresponds to intersecting the projective variety with the affine chart.

With the system reduced and stabilized, the final critical parameter is the choice of admissible monomial order. For generic systems, the graded reverse lexicographic order (*grevlex*) is empirically and theoretically optimal. By directing the algorithm to eliminate the highest-degree terms involving the last variables first, *grevlex* keeps intermediate polynomials short and coefficients small, delaying the explosion of expression swell. However, the efficiency of *grevlex* is sensitive to the specific permutation of input variables. Pre-processing heuristics can analyze the input generators to determine an optimal static ordering; a common strategy places variables appearing with high frequency or power at the bottom of the order to delay their processing until the system is partially simplified.

While *grevlex* is efficient for calculating invariants like the Hilbert series, applications requiring elimination theory, such as solving systems of equations, necessitate a Lexicographic basis. Computing a Lex basis directly is often computationally intractable due to massive coefficient growth. To circumvent this, standard workflows utilize a two-step process: first computing the basis in the fast *grevlex* order, and then converting it to the target Lex order using a change-of-ordering algorithm. For zero-dimensional ideals, the FGLM algorithm [17, 20] performs this conversion efficiently using linear algebra. For positive-dimensional ideals, the Gröbner Walk algorithm [11, 3, 18, 33] achieves the same result by traversing the Gröbner fan, moving between adjacent cones to iteratively transform the basis. This compute-then-convert pipeline is typi-

cally orders of magnitude faster than a direct Lex computation.

2.5 Complexity and Computational Bounds

To contextualize the performance improvements proposed in Section 2.4, we must first address the inherent computational complexity of constructing Gröbner bases. The behavior of Buchberger's algorithm is notoriously volatile; while it solves small systems almost instantaneously, slight increases in the number of variables or the degree of input polynomials can make the computation infeasible. We analyze complexity primarily through the time complexity (number of arithmetic operations) and the space complexity (memory requirements). In the context of Gröbner bases, both metrics are inextricably linked to the degrees of the polynomials generated during the intermediate steps of the algorithm. If the degrees of intermediate S-polynomials explode, the number of terms, and thus the matrix sizes in linear algebra steps, grows combinatorially.

The theoretical worst-case performance of Gröbner basis computation is discouraging. The Ideal Membership problem is known to be EXPSPACE-complete, as shown in [28]. Consequently, any algorithm that computes a Gröbner basis must, in the worst case, require space (and time) double exponential in the number of variables. Let d be the maximal degree of the input generators and n be the number of variables. There exists a rigorous upper bound on the degree of the polynomials in the reduced Gröbner basis, often cited as the Dubé bound [27, 4], given by

$$\deg(GB(I)) \leq 2\left(\frac{d^2}{2} + d\right)^{2^{n-1}}$$

As demonstrated in [6, 23], the degrees of the ideal can grow as $O(d^{2^n})$. To visualize why this occurs, one can imagine ideals constructed to simulate counter machines or similar computational devices. These pathological ideals force the variable degrees to act as counters that must reach astronomical values to trigger cancellations.

Example 2.5.1. Consider a simplified mechanism of degree explosion. Let $I \subset k[x_1, \ldots, x_n]$ contain relations that recursively power variables, such as

$$x_{i+1} - x_i^d \in I \quad \text{for } 1 \leq i < n.$$

In this chain, x_2 effectively represents x_1^d, x_3 represents $x_2^d = x_1^{d^2}$, and by the time we reach x_n, we are manipulating terms of degree d^{n-1}. While this specific example is a simplification, [28] constructs ideals where the relations enforce a synchronization constraint that prevents short-circuits, forcing the algorithm to traverse a path of length 2^{2^n} in the polynomial state space.

For practical purposes, this implies that if an arbitrary input system triggers this worst-case behavior, no amount of hardware optimization or parallelization will suffice. The computation will exhaust available memory long before completion. Fortunately, such pathological cases are statistically rare. In algebraic geometry, we distinguish between arbitrary systems and generic systems. A system is considered generic if its coefficients do not satisfy any specific algebraic relations that would cause accidental cancellations or dependencies. For a generic system of n equations in n variables with degrees $d_1, \ldots, d_n$, the sequence of polynomials $f_1, \ldots, f_n$ forms a *regular sequence*. This means that the algebraic variety defined by the ideal has the expected dimension of zero, and the parts at infinity behave well.

Under these conditions, the complexity is governed by the Macaulay bound [26, 29], which is significantly more manageable. The maximal degree of polynomials appearing in the computation is bounded by the index of regularity given by

$$d_{reg} = \sum_{i=1}^{n} (d_i - 1) + 1.$$

For a system where all generators have degree d, this simplifies to roughly $n(d-1)+1$. Since the number of monomials of this degree grows exponentially with n, this is merely single exponential in the input size, rather than double exponential.

Remark 1. The disparity between the worst-case and generic case is very pronounced. For a system with $n = 10$ variables and quadratic generators ($d = 2$); in the generic case, the maximum degree is approximately $10(2-1)+1 = 11$. The number of monomials of degree at most 11 in 10 variables is roughly $\binom{10+11}{10} \approx 3.5 \times 10^5$, which is computationally feasible. In contrast, in the worst-case scenario, the maximum degree could reach $2^{2^{10}}$, leading to an unmanageable number of monomials.

Even within the favorable confines of generic systems, the complexity of the computation is strictly dependent on the choice of monomial ordering. This dependency is not merely an implementation detail but a fundamental property of the algorithm. The Lexicographic Order is notoriously slow for basis construction. Because $x_1 > x_2 > \cdots > x_n$, the algorithm attempts to eliminate variables one by one. This elimination process, conceptually similar to Gaussian elimination but for non-linear terms, tends to produce polynomials with massive coefficients and extremely high degrees, often approaching the double-exponential bounds even for modest systems.

In contrast, the Graded Reverse Lexicographic Order (Grevlex) sorts primarily by total degree and breaks ties by penalizing the last variables. As proved in [5], for generic ideals, Grevlex minimizes the maximal degree of polynomials appearing in the Gröbner basis. It maintains the homogeneity of the system as long as possible, preventing the premature degree explosion seen in Lex. Because of this efficiency gap, a standard strategy in the field is to perform the heavy computational lifting using Grevlex, and then, if a Lex basis is required (e.g., for solving equations), to use a conversion algorithm like [20] or [33] to transform the result.

CHAPTER 3

POLYNOMIAL MODELS

Some models

BIBLIOGRAPHY

[1] Rafał Abłamowicz. *Some Applications of Gröbner Bases in Robotics and Engineering*, pages 495–517. Springer London, London, 2010.

[2] William W. Adams and Philippe Loustaunau. *An Introduction to Gröbner Bases*, volume 3rd. American Mathematical Society, 1994.

[3] Beatrice Amrhein, Oliver Gloor, and Wolfgang Küchlin. On the walk. *Theoretical Computer Science*, 187(1):179–202, 1997.

[4] Dave Bayer and David Mumford. What can be computed in algebraic geometry?, 1993.

[5] Dave Bayer and Michael Stillman. A criterion for detecting m-regularity. *Inventiones mathematicae*, 87(1), 1987.

[6] Dave Bayer and Michael Eugene Stillman. On the complexity of computing syzygies. *J. Symb. Comput.*, 6:135–147, 1988.

[7] Bruno Buchberger. *An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal*. PhD thesis, University of Innsbruck, 1965.

[8] Bruno Buchberger. *Ein Algorithmus zum Auffinden der Basiselemente des Restklassenringes nach einem nulldimensionalen Polynomideal*. PhD thesis, University of Innsbruck, 1965.

[9] Bruno Buchberger. Gröbner bases: An algorithmic method in polynomial ideal theory. In N. K. Bose, editor, *Multidimensional Systems Theory: Progress, Directions and Open Problems in Multidimensional Systems*, volume 16 of *Mathematics and Its Applications*, chapter 6, pages 184–232. D. Reidel Publishing Company, Dordrecht, 1985.

[10] Bruno Buchberger. *Gröbner Bases: An Algorithmic Method in Polynomial Ideal Theory*, chapter 6, pages 184–232. D. Reidel Publishing Company, Dordrecht, 1985.

[11] S. Collart, M. Kalkbrener, and D. Mall. Converting bases with the gröbner walk. *Computational algebra and number theory*, 24, 1997.

[12] David Cox, John Little, and Donal O'Shea. *Using Algebraic Geometry*. Springer, 2nd edition, 2005.

[13] David Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer, 5th edition, 2025.

[14] Persi Diaconis and Bernd Sturmfels. Algebraic algorithms for sampling from conditional distributions. *The Annals of Statistics*, 26(1):363 – 397, 1998.

[15] Cristian Eder. *Signature-based algorithms to compute standard bases*. PhD thesis, University of Kaiserslautern, 2012.

[16] Jean-Charles Faugère. A new efficient algorithm for computing gröbner bases without reduction to zero (f5). In *International Symposium on Symbolic and Algebraic Computation*, 2002.

[17] J.C. Faugère, P. Gianni, D. Lazard, and T. Mora. Efficient computation of zero-dimensional gröbner bases by change of ordering. *Journal of Symbolic Computation*, 16(4):329–344, 1993.

[18] K. Fukuda, A. N. Jensen, N. Lauritzen, and R. Thomas. The generic groebner walk, 2005.

[19] Rüdiger Gebauer and H. Michael Möller. On an installation of buchberger's algorithm. *Journal of Symbolic Computation*, 6(2):275–286, 1988.

[20] V. P. Gerdt and D. A. Yanovich. Implementation of the fglm algorithm and finding roots of polynomial involutive systems. *Programming and Computer Software*, 29(2), 2003.

[21] Alessandro Giovini, Teo Mora, Gianfranco Niesi, Lorenzo Robbiano, and Carlo Traverso. "one sugar cube, please" or selection strategies in the buchberger algorithm. In *Proceedings of the 1991 International Symposium on Symbolic and Algebraic Computation*, ISSAC '91, pages 49–54, New York, NY, USA, 1991. Association for Computing Machinery.

[22] Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. Available at https://macaulay2.com/.

[23] Jee Heub Koh. Ideals generated by quadrics exhibiting double exponential degrees. *Journal of Algebra*, 200:225–245, 1998.

[24] Martin Kreuzer and Lorenzo Robbiano. *Computational Commutative Algebra 1*. Springer Verlag, 2000.

[25] Zhiping Lin, Li Xu, and Qinghe Wu. Applications of gröbner bases to signal and image processing: a survey. *Linear Algebra and its Applications*, 391:169–202, 2004. Special Issue on Linear Algebra in Signal and Image Processing.

[26] F. S. MacAulay. Some properties of enumeration in the theory of modular systems. *Proceedings of the London Mathematical Society*, s2-26(1):531–555, 1927.

[27] Alexander Maletzky. Formalization of dubé's degree bounds for gröbner bases in isabelle/hol. In Cezary Kaliszyk, Edwin Brady, Andrea Kohlhase, and Claudio Sacerdoti Coen, editors, *Intelligent Computer Mathematics*, pages 155–170, Cham, 2019. Springer International Publishing.

[28] Ernst W Mayr and Albert R Meyer. The complexity of the word problems for commutative semigroups and polynomial ideals. *Advances in Mathematics*, 46(3):305–329, 1982.

[29] Richard P. Stanley. The upper bound conjecture and cohen-macaulay rings. *Studies in Applied Mathematics*, 54(2):135–142, 1975.

[30] Bernd Sturmfels. What is a gröbner basis? *Notices of the American Mathematical Society*, 52(10), 2005.

[31] Seth Sullivant. *Algebraic Statistics*, volume 194. Graduate Studies in Mathematics, American Mathematical Society, 2018.

[32] Y. Sun and D. Wang. The f5 algorithm in buchberger's style. *Journal of Systems Science and Complexity*, 24(6):1218–1231, 2011.

[33] Quoc-Nam Tran. A fast algorithm for gröbner basis conversion and its applications. *Journal of Symbolic Computation*, 30(4):451–467, 2000.

APPENDIX A

ALGORITHM IMPLEMENTATION

Macaulay2 code for the multivariate polynomial division algorithm and ideal membership check is provided below.

```
multivariateDivision = (h, F) -> (
R := ring h;

p := h;
r := 0_R;

while p != 0 do (
    divisible := false;
    ltP := leadTerm p;

    for i from 0 to #F-1 do (
        f := F#i;
        ltF := leadTerm f;

        if ltP % ltF == 0 then (
            factor := ltP // ltF;
            p = p - factor * f;
            divisible = true;
            break;
        );
    );
    if not divisible then (
```

```
            r = r + ltP;
            p = p - ltP;
        );
    );
    return r;
);

checkMembership = (h, F) -> (
    remainder := multivariateDivision(h, F);
    if remainder == 0 then (
        print("Result: h is in the ideal generated by F.");
        return true;
    ) else (
        print("Result: remainder is non-zero (" | toString(remainder)
        print("Note: If F is a Groebner Basis, h is NOT in the ideal.
        return false;
    );
);

-- Usage for Example 2.2.2
R = QQ[x, y, MonomialOrder => GRevLex];
F = {x^2 * y - x, x^2 + y^3};
h = x^2 * y + y^4;
checkMembership(h, F);
```